

Draft Lendas – Como o jogo foi implementado

Documento de funcionamento e implementação do protótipo

Versão documentada: 1.0.0 · Revisão: 7 de setembro de 2026

Este documento descreve o código presente no projeto. Recursos futuros aparecem identificados como propostas, separados do que já funciona.

1. O que foi construído

O Draft Lendas é um jogo de navegador em que você monta uma equipe com cinco versões históricas de jogadores profissionais de League of Legends e acompanha uma campanha simulada de Worlds. O objetivo do protótipo é testar se escolher jogadores com pools diferentes e disputar séries é divertido.

A decisão principal acontece no draft. Cada jogador traz cinco campeões, com notas próprias e ordem fixa. Você escolhe o jogador; os campeões entram automaticamente conforme o número do jogo da série.

Após montar a equipe e entrar no Worlds, é possível acompanhar acontecimentos e KDA em velocidades diferentes ou assistir apenas aos resultados rápidos. Os dois modos percorrem o torneio automaticamente até a eliminação ou o título.

O jogo funciona localmente, sem cadastro. Os dados da campanha ficam na memória do navegador: atualizar a página ou iniciar outro draft descarta a campanha anterior.

2. Como a referência visual foi aplicada

A imagem enviada orientou a identidade da interface: fundo claro, texto escuro, verde como destaque, títulos grandes, progresso por posição e imagens de campeões. A proposta foi adaptada à regra do prompt: as cartas selecionam jogadores, e cada uma apresenta o pool de cinco campeões.

A interface usa Barlow Condensed nos títulos e números em destaque, DM Sans nos textos e ícones Lucide. Fontes e imagens utilizadas pelo jogo estão disponíveis no projeto, sem precisar buscá-las em serviços externos durante uma partida.

No desktop, as opções do draft aparecem em três colunas. No celular, os cards são empilhados e colocam a imagem ao lado dos dados. As tabelas de KDA se adaptam à largura disponível. Os controles da transmissão ficam acessíveis durante a rolagem.

Botões indicam seleção, pausa e velocidade. Os diálogos podem ser fechados pelo teclado. A interface também respeita a preferência de redução de movimento do dispositivo.

3. O caminho do jogador

Momento	O que acontece	Participação do usuário
Início	Apresentação do jogo e acesso às instruções	Clicar em Começar draft
Draft	Sorteio de opções para as cinco posições	Escolher um jogador por posição
Equipe	Exibição do elenco e das composições G1 a G5	Examinar os jogos e escolher o modo de reprodução
Entrada no Worlds	Início da campanha com O vitórias e O derrotas	Clicar em Entrar no Worlds
Torneio	Confrontos e resultados avançam automaticamente	Acompanhar, pausar, mudar velocidade ou abrir relatórios
Resultado	Colocação final e histórico da campanha	Consultar os jogos ou iniciar um novo draft

Fluxo principal: início → cinco escolhas → equipe → Suíço → quartas → semifinal → final → resultado.

A campanha também pode terminar com três derrotas no Suíço ou uma derrota em qualquer série dos playoffs. Os botões de antecipação permitem encurtar os intervalos; nenhum deles é necessário para a progressão automática.

4. Como funciona o draft

As posições são escolhidas na ordem TOP, JUNGLE, MID, ADC e SUPPORT. Na tela, JUNGLE e SUPPORT também aparecem abreviadas como JG e SUP.

Ao começar um draft, a função `createDraft` prepara as cinco rodadas. Para cada posição, ela identifica os anos disponíveis, sorteia um ano, identifica as regiões daquele grupo e sorteia a região. Depois, apresenta os jogadores que correspondem à posição, ao ano e à região sorteados.

O código já considera mais de uma região, mas a base atual contém apenas a LCK. Por isso, nesta versão a região sempre será LCK. Há três alternativas por posição e ano. Os sorteios das posições são independentes, então a mesma edição do Worlds pode aparecer mais de uma vez no draft.

Um clique na carta escala o jogador e avança imediatamente. Não existe confirmação intermediária nem escolha manual de campeão. Ao concluir a quinta seleção, a interface abre a visão da equipe.

Cada versão histórica tem um identificador próprio. Por exemplo, `faker-2017-skt` e `faker-2023-t1` são entidades diferentes. O identificador combina nome, ano e equipe.

5. Pools e estratégia das escolhas

Cada jogador possui exatamente cinco campeões diferentes. A posição no pool determina quando o campeão entra:

Slot	Uso dentro de uma série
G1	Primeiro jogo: BO1, BO3 ou BO5
G2	Segundo jogo: BO3 ou BO5
G3	Terceiro jogo: BO3 ou BO5, se necessário
G4	Quarto jogo: BO5, se necessário
G5	Quinto jogo: BO5, se necessário

Uma nova série sempre reinicia no G1. O pool não é consumido entre adversários, e campeões não são bloqueados globalmente durante o torneio. Assim, o sistema se inspira na importância de um pool amplo, com uma regra própria de ordem fixa.

O jogador não possui um overall único. A força depende do campeão que ele utilizará naquele jogo. As três curvas básicas de teste são:

Perfil	G1	G2	G3	G4	G5
Explosivo no G1	99	95	89	87	85
Pool consistente	94	94	95	93	94
Decisivo no G5	89	91	94	98	99

A implementação distribui esses perfis conforme a posição e o índice do elenco na base. Alguns elencos recebem um ajuste de menos um ponto em cada slot. Os pools também usam modelos por posição, com algumas substituições por época e jogador.

Essas curvas criam escolhas para o protótipo; não são avaliações históricas verificadas da habilidade de cada profissional. Um especialista em G1 tende a ajudar nos BO1, enquanto outro perfil pode ganhar valor quando a série chega aos jogos seguintes.

6. Como a força da equipe é calculada

O cálculo ocorre para cada jogo da série e está em `src/game/engine.ts`. A configuração `BALANCE` concentra os pesos da força, os bônus de composição e os parâmetros da probabilidade de vitória.

Primeiro, o jogo calcula a média dos cinco ratings correspondentes ao slot atual. Depois, calcula uma nota de composição usando as tags dos campeões.

Critério da composição	Pontos
Nota inicial	62
Presença de AP_DAMAGE e AD_DAMAGE	+8
Pelo menos um campeão com FRONTLINE	+6

Critério da composição	Pontos
Pelo menos um campeão com ENGAGE	+5
Pelo menos um campeão com PEEL	+4
Pelo menos três ocorrências de TEAMFIGHT	+6
Pelo menos três ocorrências de SCALING	+6
Pelo menos três ocorrências de PICK	+6
Pelo menos três ocorrências de POKE	+6
Pelo menos três ocorrências de EARLY_GAME	+6

Os bônus são acumuláveis e a nota final de composição é limitada a 100. Um campeão pode contribuir com várias tags. Tags como MOBILITY, CONTROL e SPLIT_PUSH já existem nos dados, mas não recebem bônus direto nesta fórmula.

```
Força do jogo = (média dos ratings × 0,80) + (composição × 0,20)
```

O resultado é arredondado para uma casa decimal. Exemplo ilustrativo: ratings 99, 94, 89, 99 e 94 produzem média 95. Se a composição receber 91, a força final será 94,2.

Na visão da equipe, é possível alternar entre os cinco jogos e conferir a média, a composição e a força calculada para cada um. Não há bônus de estilo de jogo aplicado nesta versão.

7. Como uma partida é decidida

A função `simulateGame` calcula a força das duas equipes com a mesma regra. A diferença é convertida em chance de vitória por uma curva logística:

```
Diferença = força da sua equipe - força adversária
Probabilidade = 1 / (1 + exp(-Diferença / 6))
Probabilidade final = limitada ao intervalo de 8% a 92%
```

Equipes com a mesma força têm 50% de chance. Uma força 96 contra 93 gera aproximadamente 62,2% de chance para a primeira equipe. Portanto, a vantagem favorece a vitória, mas permite derrotas inesperadas.

O motor sorteia um número entre zero e um. Sua equipe vence quando esse número é menor que a probabilidade calculada. A função aceita uma fonte de aleatoriedade substituível, permitindo controlar resultados nos testes.

O resultado de cada jogo guarda número do jogo, vitória ou derrota, força das duas equipes, probabilidade e relatório. A vitória é definida antes de criar os acontecimentos e o KDA. O relatório apresenta uma história compatível com o resultado; ele não é a causa matemática da vitória.

8. Suíço, séries e playoffs

No Suíço, vitórias e derrotas contam confrontos vencidos ou perdidos. Vencer uma BO3 por 2–1 adiciona uma vitória à campanha do Suíço, não duas. O resultado final também informa os totais de jogos individuais, que são contados separadamente.

Campanha antes do confronto	Formato
0–0, 1–0, 0–1 ou 1–1	BO1
2–0, 2–1, 0–2, 1–2 ou 2–2	BO3

Três vitórias classificam para as quartas. Três derrotas encerram a campanha. Nos playoffs, quartas, semifinal e final são BO5. A primeira equipe a alcançar duas vitórias vence uma BO3; a primeira a alcançar três vence uma BO5.

O motor encerra a série assim que alguém atinge o número necessário. Em um 3–0, G4 e G5 ficam como não jogados. Um 2–2 em BO5 leva ao G5. Não são gerados relatórios para jogos desnecessários.

Os resultados possíveis são Eliminado no Suíço, Quartas de final, Semifinalista, Vice-campeão e Campeão mundial.

Os adversários são formados agrupando as versões dos jogadores por equipe e ano. Cada elenco adversário usa a mesma estrutura e os mesmos cálculos do elenco do usuário. O sorteio evita repetir adversários já enfrentados enquanto existirem opções novas. Não há restrição que impeça um adversário de conter versões também escolhidas pelo usuário.

O torneio acompanha apenas a campanha do usuário. Não existe uma simulação completa de 16 equipes, pareamento por campanha no Suíço ou jogos paralelos do outro lado de uma chave. A interface mostra uma rota de progresso até o título.

9. Os dois modos automáticos

Recurso	Acompanhar partida	Resultado rápido
Apresentação	Acontecimentos revelados em sequência	Resultado final resumido
KDA	Atualizado ao longo da apresentação	Disponível no relatório completo
Placar de abates	Evolui a cada acontecimento	Mostra o placar final
Avanço do torneio	Automático	Automático
Velocidades	1x, 2x e 4x	1x, 2x e 4x
Pausa e troca de modo	Disponíveis	Disponíveis

O modo detalhado é a opção inicial. A velocidade altera o tempo de apresentação, sem mudar ratings, probabilidade ou o resultado já sorteado. Um novo draft mantém a preferência de modo e velocidade e desativa a pausa.

Em `App.tsx`, um efeito controla um único temporizador por etapa. Ele prepara o confronto, gera o jogo, revela os acontecimentos, registra o placar e avança a campanha. Ao mudar o estado, o temporizador anterior é cancelado antes de programar o próximo.

Pausar, abrir instruções ou consultar um relatório suspende esse avanço. Fechar o diálogo retoma a reprodução se o usuário não tiver acionado a pausa. Mudar de modo durante um jogo reutiliza o resultado e o relatório que já foram gerados.

Tempos-base configurados, antes de dividir pela velocidade escolhida:

Intervalo	Detalhado	Rápido
Iniciar o próximo confronto	2,40 s	0,85 s
Preparar e gerar um jogo	1,40 s	0,55 s
Revelar o próximo acontecimento	2,40 s	Sem sequência; abre o momento final
Registrar o jogo após a apresentação	0,65 s	0,20 s
Intervalo após um jogo que não encerra a série	4,20 s	1,50 s
Avançar após encerrar a série	5,00 s	1,70 s

Ao mudar do detalhado para o rápido no meio da apresentação, há um intervalo-base de 0,25 segundo até exibir o último momento. Em 4×, os intervalos são divididos por quatro. São tempos programados; agendamento do navegador e pausas podem alterar o tempo real observado.

10. Acontecimentos e KDA

O relatório é gerado em `src/game/recap.ts`. Cada jogo recebe uma duração fictícia de 26 a 37 minutos e sete momentos: Primeiro sangue, Pressão no rio, Briga pelo dragão, Luta no meio, Disputa pelo Barão, A base está aberta e Nexus destruído.

Os quatro primeiros acontecem nos minutos 3, 8, 13 e 18. Os demais são posicionados em função da duração da partida. Esses minutos representam o tempo narrativo de League of Legends, e não o tempo que o usuário espera na tela.

Cada momento guarda uma cópia cumulativa do KDA dos cinco jogadores de cada lado. K significa abates, D significa mortes e A significa assistências. As cópias anteriores não são alteradas quando novos eventos são gerados.

A distribuição segue regras simples:

- O primeiro sangue adiciona um abate, sem troca adversária.
- Nos momentos seguintes, o lado destacado recebe de dois a cinco abates; o outro lado pode receber de zero a dois.
- O autor de cada abate é sorteado com peso baseado no rating do campeão e na posição. MID e ADC usam multiplicador 1,4; suporte usa 0,3 para esse sorteio; TOP e JUNGLE usam 1.

- A vítima também é sorteada por peso. Para esse sorteio, o suporte usa multiplicador 1,15; MID e ADC continuam com 1,4.
- Cada abate adiciona exatamente uma morte a um jogador adversário.
- Outros jogadores do mesmo lado podem receber assistência. O suporte tem probabilidade de 88%; os demais, 55%. O autor do abate não recebe assistência pelo próprio abate.
- Nos momentos anteriores ao fim, o vencedor da partida tem 62% de chance de ser o lado destacado. O último momento sempre pertence ao vencedor já definido.

Essas regras mantêm a contabilidade coerente, mas não constituem uma simulação de combate. Não existem movimentação, dano, tempo de renascimento, ouro, itens ou controle real de objetivos. Dragão e Barão aparecem como contextos da narrativa. Os sete tipos de acontecimento se repetem entre partidas, variando lados, jogadores, placares e parte dos tempos.

Na tela, [MatchReport](#) apresenta o relógio, o placar de abates, o acontecimento atual, as duas tabelas de KDA e a linha do tempo. Ao encerrar, mostra também resultado, forças e chance de vitória. O histórico permite abrir esse relatório depois, inclusive no modo rápido.

11. Base de jogadores e recursos visuais

A versão atual contém 45 versões de jogadores distribuídas em nove elencos e três anos. Há 44 campeões cadastrados, com identidade, imagem, splash art e tags próprias.

Ano	Elencos incluídos
2017	SKT, SSG e LZ
2020	DWG, DRX e GEN
2023	T1, GEN e KT

Os elencos representam formações históricas selecionadas. A base não pretende conter todos os participantes dessas edições. Os pools e ratings são dados fictícios de equilíbrio, e todos os slots atuais estão identificados com `source: 'MOCK'`.

O modelo já aceita `WORLDS_DATA` e `SEASON_DATA` como origens futuras. Esses campos permitirão identificar de onde veio cada associação de jogador e campeão após uma importação validada.

As artes de campeões vieram do Data Dragon e estão em `public/assets`. Não há troca automática de arte por ano histórico. Há retratos de arquivo para Faker, Huni e Zeus. Nos demais casos, o card mostra a arte do campeão G1 com legenda identificadora. Os retratos também podem ser de temporadas diferentes da versão selecionada.

Os créditos estão nas instruções do jogo, no README e em `public/assets/portrait-credits.json`.

12. Organização técnica

React monta as telas e mantém o estado da sessão. TypeScript define os contratos dos dados. Vite executa o ambiente de desenvolvimento e gera os arquivos estáticos de produção. O protótipo não usa uma API remota durante o jogo.

Arquivo ou diretório	Responsabilidade
<code>src/main.tsx</code>	Inicialização do React, fontes e estilos
<code>src/App.tsx</code>	Telas, estado da sessão, reprodução automática e diálogos
<code>src/game/types.ts</code>	Tipos de jogador, campeão, série, torneio e relatório
<code>src/game/engine.ts</code>	Draft, composição, força, probabilidade e progressão do torneio
<code>src/game/recap.ts</code>	Geração da narrativa e dos snapshots de KDA
<code>src/data/players.ts</code>	Elencos e modelos dos pools de teste
<code>src/data/champions.ts</code>	Identidade e tags dos campeões
<code>src/data/repository.ts</code>	Contrato de carregamento e adaptador local
<code>src/components/AutoplayControls.tsx</code>	Modo, velocidade e pausa
<code>src/components/MatchReport.tsx</code>	Apresentação dos acontecimentos e do KDA
<code>src/styles.css</code>	Identidade visual e adaptação das telas
<code>src/components/autoplay.css</code>	Estilos da transmissão e dos relatórios

O motor de regras recebe dados e devolve resultados sem depender de componentes React. Os cálculos de força e probabilidade ficam concentrados em `BALANCE`. Os parâmetros da narrativa estão em `recap.ts`; os tempos de reprodução estão em `App.tsx`.

Os sistemas sugeridos no prompt foram representados por funções, sem criar uma classe para cada nome: `createDraft` cuida do draft; `teamStrength` e `compositionScore`, das notas; `simulateGame`, da partida; `createSeries`, `seriesDone` e `advanceTournament`, das séries e do torneio.

O contrato `DataRepository.load()` devolve uma promessa com `{ players, champions }`. Hoje, `LocalDataRepository` retorna os dados incluídos no projeto. Na carga inicial, `validateData` verifica IDs duplicados, tamanho e ordem dos pools, campeões repetidos ou ausentes e ratings fora do intervalo permitido.

13. Preparação para Firebase ou Vercel com Supabase

A preparação entregue é uma separação de responsabilidades e dois arquivos de configuração. Nenhum projeto de nuvem, banco, autenticação ou sincronização de partidas foi criado.

`vercel.json` define Vite, comando `npm run build` e saída `dist`. `firebase.json` aponta o Hosting para `dist` e inclui o redirecionamento para `index.html`. Esses arquivos não publicam o site por conta própria.

Para carregar jogadores e campeões de um banco no futuro, será possível criar outro adaptador de `DataRepository`. Ele deverá devolver os mesmos tipos de dados e preservar a validação na entrada. Isso permite manter as regras do motor ao trocar a origem do catálogo.

Uma modelagem futura para Supabase pode separar campeões, versões de jogadores e slots do pool. Uma modelagem com Firestore pode guardar os cinco slots dentro de cada versão e manter os campeões separados. São propostas de organização, ainda não implementadas.

Salvar campanhas, retomar uma sessão ou adicionar contas exigirá contratos e funcionalidades adicionais. O repositório atual carrega o catálogo; ele não persiste o progresso do usuário. Caso sejam adicionados rankings competitivos, a validação dos resultados deverá passar por um backend confiável, pois a simulação atual é executada no navegador.

14. Verificações realizadas

Na última validação da implementação, passaram 28 testes de regras e seis testes de navegação, além da geração do build de produção. Esta revisão documental descreve essa validação já realizada; ela não acrescentou funcionalidades ao jogo.

Os testes de regras verificam os pools, as opções do draft, a fórmula de força, as probabilidades, os formatos do Suíço, os desempates, o encerramento antecipado de séries e os resultados finais possíveis.

Os testes do relatório conferem igualdade entre abates e mortes adversárias, assistências válidas, números inteiros não negativos, progressão cumulativa e ordem cronológica dos momentos. Também verificam se o Nexus é destruído pelo vencedor e se a mesma semente produz o mesmo relatório.

No navegador, os testes usam Edge em dimensões de desktop e celular. Eles cobrem uma campanha campeã sem cliques de avanço, eliminação automática no Suíço, consulta de relatórios, novo draft, atualização do KDA, pausa, troca de velocidade, mudança de modo e ausência de rolagem horizontal nos cenários verificados.

Essa cobertura verifica o funcionamento dos caminhos exercitados. Ela não comprova equilíbrio competitivo, diversão, precisão histórica nem compatibilidade com todos os navegadores; esses pontos dependem de testes adicionais e uso por jogadores.

15. Como executar e onde alterar

Na pasta do projeto, os comandos disponíveis são:

```
npm install
npm run dev
npm test
npm run test:e2e
npm run build
npm run preview
```

`npm run dev` inicia o ambiente local, normalmente na porta 5173. `npm run build` verifica TypeScript e gera a pasta `dist`. `npm run preview` permite conferir o build. Os testes de navegador pressupõem o Edge instalado, conforme `playwright.config.ts`.

Alteração desejada	Ponto principal de manutenção
Mudar a importância dos ratings ou da composição	BALANCE em <code>src/game/engine.ts</code>
Mudar bônus de sinergia e a curva de vitória	<code>src/game/engine.ts</code>
Alterar jogadores, pools, ratings ou anos	<code>src/data/players.ts</code>
Alterar tags e imagens de campeões	<code>src/data/champions.ts</code>
Ajustar acontecimentos e distribuição de KDA	<code>src/game/recap.ts</code>
Ajustar os intervalos dos modos automáticos	Efeito de reprodução em <code>src/App.tsx</code>
Alterar visual do relatório e dos controles	<code>src/components</code>
Carregar catálogo de um backend	Novo adaptador em <code>src/data/repository.ts</code>

O próximo trabalho de produto pode se concentrar em observar os drafts feitos pelos usuários, comparar as escolhas entre G1 e séries longas e coletar feedback sobre a duração da transmissão. A partir desses resultados, será possível ajustar o equilíbrio e ampliar a base histórica com dados verificados.

Documento gerado a partir do Markdown do projeto. Código de referência: versão local revisada em 07/09/2026.